Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Proje gruplarının SRS (Software Requirements Specification) belgelerinin durumu ve dönem sonu proje tanımı.
- Test yönetimi (test management) ve test spesifikasyonu (test specification) tekrarı: test planı, test senaryoları, test raporu.
- Teslim sonrası kalite: yazılım bakımı (software maintenance) ve konfigürasyon yönetimi (configuration management).
- Yazılım risk yönetimi (risk management): risk türleri, risk tablosu, riskin önemi.
- Yeniden kullanılabilirliği (reusability) artıran yaklaşımlar.
- Sürüm yönetimi (release management): ITIL çerçevesi, alt süreçler, sürüm yöneticisi, DevOps ile ilişkisi.
- Mobil uygulamalarda kalite ve test: ayırt edici özellikler, test türleri, zorluklar, gerçek cihaz ve emülatör kullanımı.

Hocanın Özellikle Vurguladığı Kısımlar

- İkinci ara sınav yapılmayacak. Onun yerine proje sunumları var. Final sınavı test (çoktan seçmeli) olarak hazırlanacak.
- Proje teslimi: yazılım test planı ve yazılım test raporu 28 Aralık'a kadar sisteme yüklenecek. Son dakikayı beklemeyin; mail ile teslim kabul edilmiyor.
- Dosyaları sisteme yüklerken zip yapmayın; gerekirse parçalara bölerek yükleyin.
- Sunumlar 29 Aralık ve 5 Ocak'ta. 29 Aralık'ta sunmak isteyen gruplar bir sonraki derse kadar hocaya mail atacak. Kalan sırayı hoca grup listesine göre belirleyecek.
- 4-5 kişilik gruplarda en az bir birim testi (unit test), bir sistem testi (system test) ve bir kullanıcı testi (user test) şart. Tek kişilik çalışmada iki test yeterli olabilir.
- Test planı her test için ayrı yapılmaz. Yazılımın bütünü için tek bir test planı hazırlanır.
- Sunumda en az bir test çalıştırılarak gösterilecek.
- Sunum dosyasında SRS'den fonksiyonel (functional) ve fonksiyonel olmayan (non-functional) gereksinimler yer alacak.
- Rapor için hazır format verilmiyor. Test planı için ilgili IEEE standardına bakılabilir.
- Sınavlarda paylaşım yapılmamalı. Herkes sınavı kendisi yapacak.
- Riskin önemi = olasılık × etki. Önemi yüksek riskler her kilometre taşında (milestone) yeniden gözden geçirilir.
- Sürüm yönetimi yalnızca geliştirme ekibinin işi değildir. İşletme tarafıyla birlikte karar verilir.
- Mobil uygulamalarda kullanıcı arayüzü (GUI) testi, kullanılabilirlik, birlikte çalışabilirlik, bağlantı ve güvenlik testleri öne çıkar. En çok hata GUI tarafında çıkar.

Kısa Tekrar Notları

- SRS belgesinde fonksiyonel gereksinimlerin yanında ayrı bir bölüm olarak fonksiyonel olmayan gereksinimler de bulunmalı.
- Test spesifikasyonu = test planı + test senaryoları + test raporu. Testleri tanımlar, planlar, düzenler ve belgeler.
- Test planı içeriği: test stratejisi, test edilecek öğeler, doğrulama yöntemleri, tamamlanma

kriterleri, hata ve sonuç raporlama, sorumluluklar, test ortamı, eğitim ihtiyacı, test takvimi, risk yönetimi, beklenen çıktılar.
- Test senaryosu içeriği: hazırlayan kişi, ilgili gereksinim veya kullanım durumu (use case), amaç, ön koşullar, girdiler, adımlar, beklenen sonuçlar.
- Test raporu: senaryoların nasıl uygulandığı, beklenen ve gerçekleşen sonuçların karşılaştırılması.
- Test ekibinin geliştirme ekibinden bağımsız olması daha tarafsız ve başarılı sonuç verir.
- Hatalar sıfırlanamaz. Amaç optimum test miktarıdır.
- Bakım türleri: hata düzeltme, iyileştirme, değişen ortama uyarlama. En sık görülen uyarlama tipidir.
- Konfigürasyon yönetiminin dört işlemi: tanımlama, değişiklik kontrolü, denetim, raporlama.
- Konfigürasyon elemanları: programlar, belgeler, veri yapıları, test verisi.
- Riskle başa çıkma: reaktif (sorun çıkınca) veya proaktif (sorun çıkmadan önleme).
- Risk sınıfları: proje, teknik, işletmecilik (iş) riskleri.
- ITIL sürüm yönetiminde altı alt süreç: destek, planlama, oluşturma ve doğrulama, dağıtım (deployment), sürüm sonrası destek, kapatma.
- Mobil test zorlukları: cihaz ve işletim sistemi çeşitliliği (fragmentation), ağ farklılıkları, güvenlik, test aracı çeşitliliği.
- Mobil testte üç seçenek: yalnızca gerçek cihaz, yalnızca emülatör, ikisinin birlikte kullanımı.

Detaylı Açıklamalar

SRS Belgeleri ve Gruplar

- Hoca grupların SRS belgelerini tek tek değerlendirdi.
- 18 Kasım'a kadar teslim eden gruplar zamanında teslim etmiş sayıldı. Geç teslim eden grup sayısı az.
- Bazı gruplar projeyi önceki derslerden aldı (yazılım mühendisliği, sistem analizi projeleri). Bu kabul edildi.
- Bazı belgelerde fonksiyonel olmayan gereksinimler eksikti.
- Fonksiyonel gereksinimler şu biçimlerden biriyle verilebilir:
  - Tüm yazılım için madde madde liste.
  - Modüllere ayrılmış liste.
  - Her modül için kullanım durumu diyagramı (use case diagram) ve belgesi.
- Bu biçimlerin hiçbiri fonksiyonel olmayan gereksinimleri içermez. Bu yüzden onlar için ayrı bölüm gerekir.
- SRS'nin baş kısmında formal bilgiler bulunur: projenin tanımı, referanslar, kullanım yeri.

Dönem Sonu Proje Tanımı

- Proje: SRS belgesi verilen yazılım için yazılım test planı ve yazılım test raporu hazırlamak.
- Test edilecek yazılım grubun kendi yazdığı yazılım olmak zorunda değil. Kitaptan, başka bir arkadaşın projesinden veya eski bir projeden seçilebilir.
- Her test için şunlar belirtilecek:
  - Test verisinin nasıl oluşturulduğu ve test verileri.
  - Kullanıcı testi için test senaryosu.
  - Testin nasıl denendiği ve sonuçları.
  - Test ortamı.
  - Beklenen değer ile gerçekleşen değerin karşılaştırması.
- Kullanıcı testi: SRS'deki hangi gereksinimin nasıl test edildiği gösterilecek.
- Birim testi: kodun dilinin birim test aracı kullanılacak. Örnek: Java için JUnit.
- Sistem testi: grupta sistem testini üstlenen kişi açık kaynak bir araç araştırıp kendi yazılımına uygulayacak.
  - Performans testi (performance test) veya yük testi (load test) yapılabilir.
  - Cevap süresi, aynı anda kaç kullanıcının bağlanabildiği gibi ölçümler yapılabilir.
  - Güvenlik testi (security test) de seçilebilir.
- Sunumda bir sunum dosyası kullanılacak. İçinde gereksinimler, plan ve test sonuçları olacak.

- Bir öğrenci kapsama (coverage) beklentisi olup olmadığını sordu (altyazıda belirsiz). Hoca beklentisi olduğunu ama bu ilk çalışma olduğu için temel şeyleri, yapılabildiği kadarını beklediğini söyledi.

Test Yönetimi ve Test Spesifikasyonu

- Test yönetimi planlama, tasarım ve gerçekleştirme adımlarıyla yürür.
- Test planı şu sorulara cevap verir:
  - Hangi tip testler yapılacak?
  - Zamanlaması ve gideri nedir?
  - Hangi ortam ve kaynaklar kullanılacak?
  - Hangi test için hangi araç (tool) kullanılacak?
- Birim testleri saydam kutu testi (white box test) olarak yapılır. Saydam kutu testinde test verisinin nasıl seçildiği belirtilir.
- Bütünlük testinde (integration test) testin tipi ve test edilen modüller belirtilir.
- Test planında risk yönetimi de yer alabilir. Risk yönetimi aslında proje yönetiminin konusudur ama kaliteyi de ilgilendirir.
- Test ekibinin görevleri:
  - Planlanan testlerin hazırlığını yapmak.
  - Testleri yürütmek.
  - Bulunan hataları yazılımcıya bildirmek ve izlemek.
  - Sonuçları raporlamak.
- "İyi bir testçi ne zaman mükemmele ulaşılacağını, iyinin yeterli olacağını bilir." Hata sıfıra indirilemez, azaltılır.
- Optimum test miktarı: hata sayısını düşüren ama maliyeti aşırı artırmayan test miktarıdır. Hoca bunu maliyet eğrisiyle hata bulma eğrisinin kesiştiği nokta olarak hatırlattı.

Teslim Sonrası Kalite: Bakım

- Geliştirme, ürünün müşteriye teslimi ve kullanıma alınmasıyla tamamlanır. Sonra bakım (maintenance) aşaması başlar.
- Bakımda yapılanlar:
  - Test aşamasında bulunamamış hataların düzeltilmesi.
  - Yazılımın yeteneklerinin veya performansının iyileştirilmesi.
  - Kurumdaki değişikliklere yazılımın uyarlanması.
- Uyarlama tipi bakım daha sık görülür. Kurumlar uygulamalarını değiştirip yazılımın buna uyarlanmasını ister.
- Bakım değişiklikleri yeniden gereksinim analizi, tasarım ve test adımlarıyla ele alınır.

Konfigürasyon Yönetimi

- Konfigürasyon yönetimi uygulanan her yazılım modülü bir konfigürasyon maddesidir (configuration item).
- Her madde için dört işlem yapılır:

İşlem	Açıklama
Tanımlama	Maddenin tanımı, genellikle bir veri tabanı uygulamasında
Değişiklik kontrolü	Maddede yapılan değişikliklerin denetimi
Denetim	Maddelerin doğruluğunun kontrolü
Raporlama	Durumun raporlanması

- Yazılım konfigürasyon elemanları: programlar, belgeler, veri yapıları, test verisi. Hoca bu sorunun sınavda da sorulduğunu hatırlattı.
- Konfigürasyon yönetimi standartlara göre yürütülmelidir.

Yazılım Risk Yönetimi

- Risk olasılık ve kayıpla ilişkilidir. Gelecekte ortaya çıkabilecek bir problemdir.

- %100 olasılıkla ortaya çıkacak bir şey risk değil, zaten var olan bir problemdir.
- Riskle başa çıkma yaklaşımları:
  - Reaktif: problem çıkınca bertaraf etmek.
  - Proaktif: problem çıkmadan önlemek.
- Risk türleri:
  - Proje riskleri: proje planını tehdit eder. Bütçe, zaman, personel riskleri.
  - Teknik riskler: yazılımın kalitesini ve zamanında bitmesini etkiler. Örnek: seçilen teknolojinin sürekliliği.
  - İşletmecilik (iş) riskleri: pazar riski, ürüne talep olup olmaması, satış stratejisi.
- Pressman'ın kitabında daha geniş bir sınıflandırma var: müşteri, işletmecilik, teknik, geliştirme ortamı, proje riskleri vb.

Risk Tablosu

- Her risk tabloda bir satırdır.

Sütun	İçerik
Risk tanımı	Riskin açıklaması
Sınıf	Teknik, personel, işletmecilik vb.
Olasılık	Örneğin %90, %50, %40
Etki	Yüksek, orta, düşük ya da sayısal aralık
Önem	Olasılık × etki

- Önem değerine göre riskler yüksek, orta gibi sınıflanır.
- Önemi yüksek riskler her kilometre taşında yeniden değerlendirilir: olasılık değişti mi, etki aynı mı?
- Her önemli risk için önlem belirlenir ve önlemin maliyeti değerlendirilir.
- Örnek: ekibin başındaki yazılım mühendisinin işten ayrılma riski. Önlem olarak yanına yardımcı alınabilir veya sözleşme yapılabilir. Bunun da bir maliyeti vardır.
- Ele alınan her risk için ayrı bir takip tablosu tutulur.

Yeniden Kullanılabilirliği Artıran Yaklaşımlar

- Nesneye yönelik programlama (object-oriented programming).
- Bileşen tabanlı yazılım geliştirme (component-based development).
- Tasarım kalıpları (design patterns).
- API ve program kütüphaneleri (libraries).
- Kod üretimi (code generation).
- Cephe yönelimli yaklaşım (aspect-oriented): yazılıma farklı bakış açılarıyla bakarak daha iyi gereksinim analizi.
- Çerçeveler (frameworks).
- Servis yönelimli mimari (service-oriented architecture).
- Yazılım ürün hatları ve ürün aileleri (software product lines).
- Hoca kalite ölçütlerine (McCall, nesneye yönelik ölçütler) önceki derslerde değinildiği için yeniden girmedi.

Sürüm Yönetimi (Release Management)

- Sürüm yönetimi konfigürasyon yönetimiyle ilişkili bir konudur.
- Yeni bir yazılımın, sistemin veya servisin ilk kez kullanıma alınması ya da yenilenmesidir.
- Karar verilen konular:
  - Yayınlama hangi gün ve saatte yapılacak?
  - Kullanıcılar eski üründen yeni ürüne nasıl geçecek?
  - Yeni sistem çalışan (canlı) ortamla nasıl bütünleşecek ve canlı ortam nasıl korunacak?
- Bazı kurumlarda sürüm yönetimini teknik ekip, bazılarında iş kurallarına uyumu sağlayan başka bir grup yürütür.
- Süreç IT Infrastructure Library (ITIL) çerçevesine göre tanımlanır.
- Sürüm, geliştirme ortamı ile işletme ortamının ortak kararlarıyla yürür. Sürüm yönetimi kullanıcılar ile proje ekibi arasında köprüdür.

- Kurumda birden çok proje varsa sıralama önemlidir. Hangi uygulama önce devreye alınmalı, ona bağlı uygulama ne zaman alınmalı?
- Uygulamalar arasında kaynak ve bilgi transferi de sıralamayı etkiler.

ITIL Sürüm Yönetimi Alt Süreçleri

Alt süreç	Açıklama
Destek (Release Management Support)	Tüm adımlara rehber ve destek sağlar
Planlama (Release Planning)	Kapsam, politika, paketlerin sırası
Oluşturma ve doğrulama (Release Build)	Sürümün hazırlanması, tüm bileşenlerde geçerlemenin denetimi
Dağıtım (Release Deployment)	Üretim ortamına geçiş, son kullanıcı eğitimi ve belgeleri
Sürüm sonrası destek (Early Life Support)	Kullanıma alındıktan sonra belli süre koruma; çıkan hatalar hemen düzeltilir
Kapatma (Release Closure)	Formal kapanış, belgelerin doğrulanması, raporlama, proje ekibine geri besleme

- Proje yönetiminde de benzer adımlar var: başlatma, planlama, yürütme, kapatma.
- Kapatmada arşiv oluşturulur: süre, iş gücü, maliyet, çıkan riskler, kullanılan teknolojiler. Bu bilgiler sonraki projelerde tahmin için kullanılır.

Sürüm Yöneticisi (Release Manager)

- Sürümün planlamasını ve koordinasyonunu yapar. Yeni sürümün etkin ve güvenli kullanıma alınmasını sağlar.
- Tipik olarak bir bilgi teknolojileri uzmanıdır.
- Yazılım geliştirme standartlarını, IT süreçlerini, iş süreçlerini ve araçları bilmelidir.
- IT ekibi ile işletme tarafındaki görevlileri bir arada çalıştırır. Hoca bunu “iyi bir orkestra şefi” olarak tanımladı.
- Pazarlama, araştırma ve geliştirme ekipleriyle koordineli çalışır.
- Birden fazla uygulama varsa diğer projeleri de bilmelidir.
- Kurumun kalite grubuyla yakın çalışır. Kurumun standartlarına uymalıdır.

Konfigürasyon Yönetimiyle İlişki ve Faydalar

- Kullanıma alınan ürünün alt elemanları, test verisi, veri tabanı gibi bilgiler konfigürasyon yönetiminde tutulur. Sürüm sırasındaki işlemler de konfigürasyon yönetimine aktarılır.
- Faydalar:
  - Yeni özellikler kullanıcılara daha hızlı ve daha az riskle ulaşır.
  - Kurulum ve kullanıma alma iş planları önceden tahmin edilir ve iyi planlanır.
  - Ürün veya hizmet gereksinimlere uygun biçimde kullanıma alınır.
  - Kullanıcılara ve destek personeline uygun bilgi transferi sağlanır.
- DevOps (Development + Operations): geliştirme ve operasyon işlerinin aynı mantık ve standartla yürütülmesi. Özellikle çevik (agile) yaklaşımda kullanılır ve sürüm yönetimiyle yakından ilişkilidir.

Mobil Uygulamalarda Kalite ve Test

- Hoca bu bölümü geçen yılın bir yüksek lisans öğrencisinin sunumu üzerinden anlattı.
- Nesneye yönelik yazılımlarda nesne özelliklerini ölçen ölçütler (CK metrikleri) öne çıkıyordu. Benzer biçimde mobil yazılımlarda da mobil cihaza özgü özellikler testte ağır basmalıdır.
- Mobil testte iki süreç var: yazılımı hazırlarken test ve kullanıma alındıktan sonra ortama uyum (adaptasyon).
- Mobil test diğer uygulamalara göre daha pahalı ve zaman alıcıdır.
- Emülatörlerle test ucuzdur ama güvenilirliği sınırlıdır.
- Kapsamlı test yapılmadan sürüme geçilmesi kullanım sırasında risk oluşturur.

- Uygulama türleri: mobil web uygulamaları, yerel (native) uygulamalar, hibrit (hybrid) uygulamalar.

Başarılı Mobil Uygulamanın Ayırt Edici Özellikleri

- Bağlantı (connectivity).
- Uygunluk ve kaliteli tasarım.
- Desteklenen cihazlar.
- Kullanılan programlama dilinin özellikleri.
- İçerik bilgisi ve güvenlik.
- Kişiselleştirme kolaylığı.
- Ulaşılabilirlik (accessibility).
- Yerelleştirme (localization).

Öne Çıkan Test Alanları

- Kullanılabilirlik (usability).
- Birlikte çalışabilirlik (interoperability): aynı uygulama farklı platformlarda test edilir.
- Mobil bağlantı: farklı ağlarda ve bağlantı senaryolarında işlevsel test.
- Güvenlik:
  - Cihazda saklanan kişisel ve ticari bilgilerin gizliliği; şifreleme ve şifre çözme teknikleri.
  - Saldırılar SMS, MMS, Wi-Fi, Bluetooth, web tarayıcısı ve işletim sistemindeki açıklardan gelebilir.
  - Şifre veya kullanıcı adının cihazda şifrelenmeden bırakılması tehdidi.
  - Kurulumda ve sonradan değiştirilebilen uygulama izinleri. Test case'ler bu senaryoların hepsini kapsamalıdır.
- Uygulamanın yapısal testi: saydam kutu (white box) testi.
- Otomatik tarama testi araçları hataları ve cihaz tutarsızlıklarını hızlı bulur.
- Yeni kapsama kriterleri mobilite algılamasını ve enerji yapılarını hesaba katmalıdır.

Mobil Test Süreci ve Test Türleri

- Test süreci: birim testi, bütünlük testi, sistem testi, kullanıcı arayüzü testi, regresyon testi (regression test), kabul testi (acceptance test).
- Kullanıcı arayüzü testi mobilde çok önemli olduğu için ayrı bir test olarak eklenir. Sistem testiyle paralel veya sonrasında yapılır.
- Durum yaklaşımlı test: farklı modlarda davranış. Örnek: düşük pil, toplantı modu, uçuş modu.
- Test türleri: fonksiyonel test, performans testi, güvenlik testi, kullanılabilirlik testi, yük ve stres testi (load/stress test), yerelleştirme testi, senkronizasyon testi.
- Kullanıcı arayüzü testinde uygulamanın farklı ekran tiplerinde, markalarda ve modellerde doğru görüntülenip görüntülenmediği test edilir. Bunu hızlandıran otomatik araçlar vardır.
- Performans; cihazın donanım gücüne, çalışma moduna, bağlantı kalitesine ve taşıyıcıya (operatör) bağlıdır. Bölgeye göre farklı cihaz ve taşıyıcılarla test edilmelidir.

Cihaz Çeşitliliği ve Test Seçenekleri

Seçenek	Artı	Eksi
Yalnızca gerçek cihaz	Daha sağlıklı sonuç	Maliyetli
Yalnızca emülatör	Ucuz, hızlı	Sınırlamalar, farklı sonuçlar
İkisinin birlikte kullanımı	Dengeli	İki ortamı yönetmek gerekir

- Sunumdaki bilgiye göre testlerin %30-40'ı emülatörlerle yapılıyor.
- Mobil bulut çözümleri: gerçek ağlara bağlı gerçek cihazlara web arayüzünden erişilerek test yapılır.

Mobil Testin Zorlukları

- Uygulama hangi cihazları ve işletim sistemi sürümlerini destekleyecek?
- Platformlar arası farklara nasıl uyum sağlanacak?
- Sık güncellenen cihaz ve teknolojilere testler nasıl yetişecek?
- Test matrisi her faktörle büyür ve karmaşıklaşır.
- Parçalanma (fragmentation): çok sayıda cihaz ve işletim sistemi, farklı arayüzler.
- Ağ: her operatör farklı destek verir. Bütün ağlarda test gerekir.
- Güvenlik: kişisel bilgilerin korunması.
- Test aracı çeşitliliği: çok sayıda açık kaynak araç gözden geçirilmeli.
- Hoca öğrencilerin de sistem testleri için bu konulara bakarak araç aramasını istedi.

Ders Sonundaki Sorular ve Duyurular

- Bir öğrenci birinci ara sınavın sınıf ortalamasını sordu. Hoca bakmadığını, sonraki hafta bakıp söyleyebileceğini belirtti.
- Sınav kağıtlarıyla ilgili gelen mailler incelendi ve notlar girildi.
- Bir sonraki hafta (22 Aralık) hoca ders anlatacak. Ondan sonraki iki hafta proje sunumları yapılacak.
- Gruplar listede yukarıdan aşağıya, yarısı ilk hafta, yarısı ikinci hafta sunacak.

Ders sonrası notlar

- Hoca riskin önemini "olasılık × etki" olarak verdi. Literatürde bu değer risk maruziyeti (risk exposure) olarak geçer: $RE = P \times C$. Burada P olasılık, C riskin gerçekleşmesi durumundaki maliyettir.
- Optimum test miktarı tam olarak toplam maliyetin (test maliyeti + bulunamayan hataların maliyeti) en düşük olduğu noktadır. Klasik grafikte bu nokta iki eğrinin kesişimi civarında gösterilir.
- Hoca "mobil uygulamalar genellikle server side teknolojisiyle oluşturulur" dedi. Bu ifade mobil web uygulamaları için doğrudur. Yerel (native) uygulamalar cihaza kurulur ve platformun kendi geliştirme araçlarıyla yazılır. Hibrit uygulamalar ikisini birleştirir.
- DevOps çevik (agile) yaklaşımla birlikte anılır. Altyazıda "Ecel" diye geçen ifade "agile" kelimesidir.
- Altyazıda "Makkol" diye geçen McCall kalite modelidir. "Presman" ise Pressman'ın yazılım mühendisliği kitabıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.